PRONOS

Web3 Prediction Markets Platform

Product Requirements Document (PRD)
Version 2.0 - April 2026
Confidential

> **About this document**
>
> This PRD describes the full PRONOS vision, what has already been built internally, and what we need from an external development team to reach mainnet. Sections clearly mark what already exists (`BUILT`) versus what still needs to be delivered (`TO BUILD`).
>
> The document intentionally keeps strategic roadmap items such as UMA and MoonPay in scope even when they are not yet implemented. Some items are launch-critical, while others are included as optional or nice-to-have future work so they remain visible in planning conversations.

---

01. General Information and Scope

Project Name

PRONOS - The first on-chain prediction market focused on Latin America.

Vision

Build the most accessible prediction platform for the Latin American market, where anyone can bet on sports, politics, and culture without needing crypto knowledge, complex wallets, or currency conversion expertise.

Launch Goal

Mainnet live before the 2026 FIFA World Cup (June 14, 2026).
Allocated budget: **$30,000 USD**.

Current Product URL

https://pronos.io/mvp/ - password protected staging environment.

Blockchain Network: Arbitrum (Ethereum Layer 2)

The selected deployment network is **Arbitrum**, Ethereum's optimistic rollup Layer 2.

Criterion	Details
Gas	About $0.01 USD per transaction, which makes micro-bets viable
Speed	Confirmation in about 250ms
Compatibility	Fully EVM-compatible, so OpenZeppelin contracts do not need to be rewritten
Ecosystem	Native USDC (Circle), full Privy support, Safe multisig available
Liquidity	Second-largest L2 by TVL (about $20B+) with broad USDC availability

Note: The original draft proposed Base because of MXNB availability through Bitso. The plan was later migrated to Arbitrum because of a more mature ecosystem, better tooling support (Safe SDK, Privy), and direct USDC availability as collateral.

Collateral Asset

Parameter	Specification
Primary asset	USDC - Circle stablecoin deployed natively on Arbitrum
Multi-asset support	Not required in v1. The architecture can support adding assets such as MXNB or DAI without redeploying core contracts

On-Ramps (Funding Methods)

Several deposit paths are planned so users can fund the platform easily:

Path	Method	Status	Priority
MoonPay	Apple Pay, Google Pay, credit and debit cards	TO BUILD	High - MVP
SPEI	MXN bank transfer via Bitso or a similar provider	TO BUILD	High - MVP
Native bridge	Arbitrum Bridge for crypto-native users	BUILT (link only)	Medium
Direct Bitso flow	Buy USDC in Bitso and send it to the wallet	Under evaluation	Post-MVP

Target flow for non-crypto users:
Open app -> log in with email -> deposit with card / Apple Pay / SPEI -> place a prediction. Total time under 3 minutes.

MoonPay handles the full KYC process (ID verification with national ID or passport), payment processing, and delivery of USDC directly into the user's Arbitrum wallet. The user never needs to understand blockchain mechanics.

---

02. Smart Contract Architecture

BUILT - Contract System

Custom contracts were built from scratch using **Solidity 0.8.24**, **Foundry**, and **OpenZeppelin v5.6.1**. The architecture is original and optimized for binary markets with dynamic fees.

`PronosToken.sol` - Conditional Tokens (ERC-1155)

Each market creates two tokens: **YES** (`marketId * 2`) and **NO** (`marketId * 2 + 1`).

- ERC-1155 standard for gas-efficient multi-market support
- Only authorized pools (minters) can mint or burn tokens
- `mintPair()`: deposit collateral and receive YES + NO in a 1:1 ratio
- `burnPair()`: return YES + NO and recover collateral
- `burn()`: burn winning tokens during redemption
- Ownership transferred to `MarketFactory` after deployment

`MarketFactory.sol` - Market Factory

This is the central hub for creating and managing prediction markets.

- `createMarket()`: deploys a new AMM pool with seed liquidity
- `resolveMarket()`: declares the winning outcome (YES = 1, NO = 2)
- `pauseMarket()`: suspends or resumes trading
- `distributeFees()`: distributes accumulated fees under the 70 / 20 / 10 split
- Access control: **owner** (Safe multisig) and **resolver** (can be a separate multisig)
- Configurable addresses: treasury, liquidity reserve, emergency reserve, fee collector

`PronosAMM.sol` - Automated Market Maker (CPMM)

An AMM using the **constant product formula (`x * y = k`)** with dynamic fees.

- **Base formula:** `x * y = k`, where x and y are YES / NO reserves
- **Buy:** user deposits USDC, the pool mints YES + NO pairs, and the CPMM computes output tokens

- **Sell:** user returns tokens, a quadratic formula computes collateral out, and the pool burns pairs
- **Redeem:** after resolution, winning tokens are redeemed 1:1 for USDC
- On-chain estimate functions: `estimateBuy()`, `estimateSell()`, `priceYes()`, `priceNo()`
- All market logic lives on-chain, with no off-chain matching engine

Testing

Metric	Status
Total tests	**43 passing**
`PronosToken` coverage	100%
`MarketFactory` coverage	87%
`PronosAMM` coverage	82%
Full cycle test	Built: create -> fund -> trade -> resolve -> redeem
Framework	Foundry (`forge test`)

Dynamic Fee Model

The fee formula adjusts automatically according to implied market probability:

```
fee% = 5 * (1 - P)
```

Where P is the implied probability of the side being bought.

Market probability	Fee per trade
50 / 50 (maximum uncertainty)	2.5%
70 / 30	1.5%
90 / 10	0.5%
99 / 1 (nearly decided)	0.05%

Principle: charge more when risk and uncertainty are higher, and less when the market has already formed a strong consensus. Fees are deducted **before** funds enter the pool and are sent directly to the `feeCollector`; they never touch AMM reserves.

Revenue Distribution

Destination	Percentage
Treasury (operations and development)	70%
Liquidity fund (LP incentives)	20%
Reserve fund (security / emergencies)	10%

Market Types

Type	Status	Description
Binary (Yes / No)	Built	Covers sports, politics, and culture. Example: "Will Mexico beat Argentina?"
Categorical (multiple outcomes)	Planned for v2	Example: "Who wins La Casa de los Famosos?" with 5+ candidates. Requires AMM extension
Scalar (numeric range)	Planned for v2	Economic markets such as exchange rate or inflation

TO BUILD - Contracts

Item	Detail
Deploy to Arbitrum Sepolia	Script is ready (`DeployProtocol.s.sol`), but still needs RPC and deployer execution
Deploy to Arbitrum One	Same script, with mainnet RPC and Arbiscan verification
Fuzz testing	AMM edge cases such as extreme amounts, reentrancy, overflow
UMA integration	Connect market resolution to UMA Optimistic Oracle
Categorical markets	Extend the AMM to support 3+ outcomes

---

03. Resolution and Oracles

Dual Resolution Mechanism

The platform will use **two different resolution mechanisms** depending on the market category:

Category	Mechanism	Rationale
Sports / Reality TV (Liga MX, FIFA, Exatlon, La Casa de los Famosos)	Custodial multisig oracle (2-of-3)	Results are public and easy to verify via broadcasts and official sources
Politics / Economy (elections, legislation, appointments, macro data)	UMA Optimistic Oracle	Handles arbitrary real-world events with on-chain dispute resolution
Special markets	Case by case	Multisig or UMA depending on event complexity

BUILT - Multisig Resolution

Component	Specification
Type	Gnosis Safe multisig (2-of-3 or 3-of-5)
Signers	Founder, co-founder, trusted third party
Process	Resolver proposes a result, required signatures are collected, then `resolveMarket()` executes
Tooling	Safe SDK integrated into the admin panel (`protocol-kit` + `api-kit`)
Supported chains	Arbitrum Sepolia and Arbitrum One
Admin UI	Create Safe, connect existing Safe, propose / sign / execute directly from `/mvp/admin`

TO BUILD - UMA Optimistic Oracle

UMA should be integrated for political and complex markets where resolution is not obviously mechanical.

Component	Requirement
UMA interface	Adapter contract connecting `MarketFactory` to UMA `OptimisticOracleV3`
Result proposal	Anyone can propose an outcome by posting a USDC bond
Dispute window	48 hours; if no one disputes, the result stands
Escalation	If disputed, it escalates to UMA's DVM, where UMA token holders vote on-chain
Auto-resolution	After the dispute window ends without challenge, `resolveMarket()` runs automatically
Resolution rules	Rules are defined when the market is created, using UMA ancillary data

Note: UMA interfaces and comments are already scaffolded in the contract code. What remains is the adapter contract and callback wiring.

Dispute Process

Oracle	Process
Multisig (sports / TV)	If an error is detected, signers deliberate and correct it by majority vote. The curation committee is the final arbiter
UMA (politics)	Any token holder can dispute during the dispute window. The dispute escalates to UMA's DVM, and the majority determines the final result

Ambiguous cases such as cancellations, technical ties, or postponed events must be handled according to the pre-defined resolution rules stored with the market.

Execution Timelines

Phase	Multisig	UMA
Reporting period	24h after event	24h after event
Dispute period	Not applicable	48h after proposal
Resolution with no dispute	24-48h	Automatic at dispute-window close
Resolution with dispute	24-48h (multisig deliberation)	5-7 business days (DVM voting)
Fund release	Immediate after resolution	Immediate after resolution
Oracle fee	None	About 1.5% of market value, paid by the protocol

---

04. Frontend and User Experience

BUILT - Technical Stack

Layer	Technology
Framework	React 18 + Vite 5 (SPA, mobile-first)

Layer	Technology
Routing	React Router v6 with basename `/mvp`
Auth / Wallet	Privy (`@privy-io/react-auth`)
Blockchain library	`ethers.js` v5.7.2
Styling	Custom CSS, proprietary design system, dark theme, responsive
Monitoring	Sentry (`@sentry/react`, `ErrorBoundary`, privacy-safe)
Hosting	Vercel (static + serverless functions)
Language	Spanish only

BUILT - Pages and Components

Home (`/mvp/`)

- Hero carousel with featured markets
- Market grid with live data (Polymarket Gamma API + internal markets)
- "How it works" visual section
- Animated market ticker
- Search bar and filtering
- Footer links
- Responsive tablet and mobile layouts

Market Page (`/mvp/market?id=`)

- Ring-chart probability visualization
- Trading panel with outcome and amount selection
- Payout preview with real-time dynamic fee calculation
- Tabs for Rules, Market Context, Comments, Top Holders, Positions, and Activity
- Sticky sidebar for quick trading
- Support for resolved markets with final winner state

Portfolio (`/mvp/portfolio`)

- Chain-aware user USDC balance
- Open positions list with individual P&L
- Summary of total invested, current value, and profit / loss

Admin Panel (`/mvp/admin`)

- Restricted access with server-side auth through `/api/user`
- Protocol mode toggle (Polymarket <-> Internal protocol)
- Market creation form (question, category, date, oracle, liquidity)

- Market list with pause and resolve actions
- Fee-formula and 70 / 20 / 10 distribution display
- Deployed-contract status panel
- Full Safe SDK integration for create / connect / propose / sign / execute
- Non-admin users receive a 404 page, making the route undiscoverable

Bet Slip (Trade Modal)

- Quick amount selectors ($5, $10, $25, $50) plus custom entry
- Dynamic calculation of fee, estimated payout, potential profit, and implied probability
- Auto network switching when user is on the wrong chain
- Full flow: check balance -> approve USDC -> sign -> place order
- Unauthenticated users see a "Join the waitlist" action via Tally

BUILT - Authentication and Onboarding

Capability	Status
Email login	Built via Privy
Google login	Built via Privy
Wallet login (MetaMask, Coinbase Wallet, etc.)	Built via Privy
Auto-created embedded wallet (ERC-4337)	Built and invisible to user
Username registration	Built with "Skip" option and auto-generated fallback
Persistent sessions	Built via Privy embedded wallets
Multi-chain	Polygon + Arbitrum + Arbitrum Sepolia
Auto network switch	Built
Chain-aware USDC balance	Built
Deposit link	Built according to active protocol mode

BUILT - Dual Mode: Polymarket + Internal Protocol

The platform currently operates in two modes, controlled from the admin panel:

Mode	Description
Polymarket	Shows Polymarket markets from Gamma and routes trades through the Polygon CLOB
Internal protocol	Shows admin-created markets and routes trades through `PronosAMM` on Arbitrum

The switch is reactive across the app through a custom event. At the current stage, Polymarket mode is used to validate UX on real markets while the internal contracts are being prepared for deployment.

TO BUILD - Frontend

Item	Detail	Priority
Connect frontend to internal contracts	`contracts.js` library for `PronosAMM` buy / sell / redeem	High
Buy / sell panel for internal protocol	Detect market type and route to the right AMM flow	High
Real-time AMM pricing	Read `priceYes()` and `priceNo()` from contracts and show them in UI	High
Slippage preview	Use `estimateBuy()` and `estimateSell()` before confirming trade	High
Unified portfolio	Merge Polymarket and internal protocol positions into one view	Medium
Historical price chart	Line or OHLC chart using `price_snapshots` table	Medium
MoonPay widget	Embedded card / Apple Pay / Google Pay funding flow	High
Push notifications	Market opening and resolution alerts via WhatsApp / SMS / push	Low
Gas sponsoring	Turn on Privy paymaster so users do not pay gas directly	Medium
Final responsive pass	Mobile and tablet review	Medium
PWA	Service worker and app-like mobile install experience	Low

Full Visualization Vision

Component	Status	Description
Real-time probability visualization	Built (ring chart)	Visual chart showing YES / NO movement from market open
Price history	To build	Hourly / daily OHLC view for market trend analysis
Position panel (P&L)	Built (basic)	Dashboard with active markets, invested amount, current value, unrealized P&L
Active market feed	Built	Filterable list by category, volume, and close date

Component	Status	Description
Individual market view	Built	Detail view with rules, oracle, activity, comments
Push notifications	To build	Alerts for market open, near resolution, and final result
Leaderboard	To build	Public ranking by user P&L and volume

---

05. Backend and Infrastructure

BUILT - Serverless API (Vercel Functions)

Endpoint	Function
`GET /api/markets`	List internal protocol markets with current price
`GET /api/market?id=`	Market detail plus 50 price snapshots, 20 recent trades, and total volume
`GET /api/positions?address=`	User positions with P&L calculation for active and resolved markets
`GET /api/user?privyId=`	User profile plus admin flag, evaluated server-side
`GET /api/indexer`	On-chain event indexer (Vercel Cron, every minute)
`GET /api/migrate?key=`	Database migration endpoint protected by key
`GET /api/bitso?action=`	MXN <-> USDC quote stub prepared for a real integration

BUILT - Database (Neon PostgreSQL)

Table	Purpose
`users`	Registered users (`privyId`, username, admin flag)
`protocol_markets`	Internal protocol markets (`chain_id`, `pool_address`, question, status, outcome)
`trades`	Indexed trade history, deduplicated by `tx_hash + log_index`
`positions`	Materialized user positions by market
`price_snapshots`	AMM price snapshots for charts

Table	Purpose
`indexer_state`	Last processed block by chain

There are six indices for fast queries, and migrations can be run through a protected endpoint.

BUILT - On-Chain Indexer

The indexer reads blockchain events and writes data into PostgreSQL:

- `MarketCreated` -> register market
- `SharesBought` / `SharesSold` -> store trade and update user position
- `MarketResolved` -> update market status
- Price snapshots -> read `reserveYes()` / `reserveNo()` and derive CPMM pricing

This runs through Vercel Cron every minute or via an authenticated manual trigger.

BUILT - Monitoring

Tool	Use
Sentry	Frontend error tracking with privacy-safe setup
Structured logger	JSON logging in API routes with duration, CORS, and stack traces

TO BUILD - Backend

Item	Detail	Priority
Real Bitso API	Replace stub with production MXN <-> USDC quotes and SPEI funding integration	Medium
MoonPay webhook	Receive deposit confirmations and update DB state	High
Rate limiting	Protect public endpoints from abuse	High
Input validation	Sanitize all user-provided inputs in API routes	High
CSRF protection	Add tokens to POST endpoints	Medium
Leaderboard API	Public ranking endpoint by P&L / volume	Low

---

06. Security

BUILT - Security Headers

Header	Value	Status
`X-Frame-Options`	`DENY`	Built
`X-Content-Type-Options`	`nosniff`	Built
`Referrer-Policy`	`strict-origin-when-cross-origin`	Built
`Permissions-Policy`	`camera=(), microphone=(), geolocation=(), payment=(), usb=()`	Built
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload`	Built
`CORS`	Restricted to `pronos.io` and localhost	Built
`X-XSS-Protection`	`0` (disabled, replaced by CSP)	Built

Internal Security Audit - Findings

Resolved

- C1: Admin authentication moved server-side
- C4: Admin allowlist removed from frontend bundle
- H2: `X-Frame-Options: DENY`
- M1-M5: Security headers completed plus restrictive CORS

Still Open

ID	Severity	Description
C2	Critical	CLOB credentials are passed in POST body instead of being derived server-side
C3	Critical	`DATABASE_URL` is reachable from frontend API layer; infrastructure tiers need separation
H1	High	Missing Content-Security-Policy
H3	High	`/mvp/admin` still lacks fully hardened server-side auth
H4	High	`/api/user` can be enumerated without authentication
M6	Medium	No SRI on external scripts

ID	Severity	Description
M7	Medium	No CSRF protection on POST requests
M8	Medium	`localStorage` acts as the source of truth for protocol mode
M9	Medium	Vite dev proxy points to production
M10	Medium	`ethers.js` v5.7.2 is outdated; upgrade to v6

Admin Roles and Multisig

Role	Control	Implementation
Owner (Safe 3-of-5)	Pause contracts, create markets, distribute fees, update protocol addresses	Gnosis Safe on Arbitrum
Resolver (Safe 2-of-3)	Resolve markets and set the winning outcome	Same Safe or a separate Safe
Admin UI	Web management through `/mvp/admin`	Server-side auth, username-based

There is **no arbitrary upgradeability**. Any change in contract logic requires multisig approval. Contracts are intended to be immutable, so logic changes require a new deployment and migration.

TO BUILD - Security

Item	Detail	Priority
Resolve C2 and C3	Separate CLOB credentials and DB access from frontend-exposed layers	Critical
CSP headers	Add a restrictive Content-Security-Policy	High
Complete admin auth	Add true server-side middleware, not just username checks	High
Rate limiting	Across all API routes	High
Contract fuzz testing	Reentrancy, overflow, and extreme-amount edge cases	High
External audit	Specialized third-party audit before high-value mainnet use	Post-launch

Item	Detail	Priority
Bug bounty	Launch a public bug bounty after go-live	Post-launch

---

07. Market Curation

v1 - Admin-Managed Markets

In v1, market creation is restricted to administrators with access to the control panel. User-generated markets are planned for v2 only.

BUILT - Admin Panel

- Create market: question, options, closing date, oracle, initial liquidity, and resolution rules
- Pause / resume market when unexpected events happen
- Resolve market by selecting the winning outcome
- Display fee metrics, distribution, and contract status
- Safe integration for propose / sign / execute directly in UI

TO BUILD

Item	Detail	Priority
Wire forms to contracts	Connect market creation form to `MarketFactory.createMarket()` via Safe	High
Wire resolve / pause	Make admin buttons execute Safe multisig transactions	High
Metrics dashboard	Volume, participants, accumulated fees, and liquidity by market	Medium
User management	Pause accounts for suspicious behavior	Low
User-generated markets	Allow moderated market proposals in v2	Post-MVP

Launch Market Set (Target: FIFA World Cup 2026)

Category	Example markets
FIFA World Cup	Will Mexico reach the Round of 16? Who wins the group? Will Argentina repeat?
Liga MX	Who wins Clausura 2026? Relegation outcome?

Category	Example markets
Mexico politics	Midterm elections, reforms, appointments
Reality TV	La Casa de los Famosos, Exatlon, La Academia
Crypto / Economy	Bitcoin price, exchange rate, inflation

The launch plan is to curate **5 to 10 markets** with seed liquidity in USDC.

---

08. Roadmap and Current Status

Phase 1: Foundation and Contracts - About 85% Complete

Item	Status
Hybrid architecture defined (Polymarket + internal protocol)	Built
Core contracts: `PronosToken`, `PronosAMM`, `MarketFactory`	Built
43 Foundry tests passing	Built
Reproducible deployment script (`DeployProtocol.s.sol`)	Built
Safe SDK integrated in admin panel	Built
Arbitrum Sepolia config in `foundry.toml`	Built
Deploy to Arbitrum Sepolia	To build
Create Safe multisig in testnet	To build
Transfer ownership to Safe	To build

Phase 2: Features and UI - About 75% Complete

Item	Status
Full frontend (Home, Market, Portfolio, Admin)	Built
Polymarket integration (Gamma + CLOB)	Built
Admin panel with market CRUD + Safe SDK	Built

Item	Status
Wallet and onboarding (Privy, multi-chain, auto-switch)	Built
Backend: schema, indexer, APIs, price snapshots	Built
Monitoring: Sentry and structured logging	Built
Bet slip with dynamic fee and preview	Built
Responsive tablet + mobile	Built
OG metadata, Twitter cards, favicon	Built
Waitlist gate (Tally)	Built
Connect frontend to internal contracts	To build
Wire admin forms to contracts via Safe	To build
Integrate MoonPay	To build
Integrate UMA Oracle	To build
Load 5 to 10 curated markets	To build
End-to-end testing in testnet	Blocked by deploy

Phase 3: Hardening and Mainnet - Target: May 31, 2026

Item	Priority
Deploy to Arbitrum One (mainnet)	Critical
Verify contracts on Arbiscan	Critical
Resolve critical and high security findings (C2, C3, H1, H3, H4)	Critical
Seed liquidity for launch markets	Critical
World Cup 2026 markets ready	Critical
48-hour stability test with no intervention	High
Technical documentation and runbook	High
Rate limiting + input validation	High
Final mobile responsive check	Medium

Post-MVP - Full Product Vision

Item	Description	Estimate
Direct Bitso	MXN on / off ramp via SPEI with in-app USDC buy / sell	$10K - $15K
Categorical markets	3+ outcomes in extended AMM	$10K - $15K
Scalar markets	Numeric-range markets like FX and inflation	$10K - $15K
External audit	Third-party audit firm	$25K - $50K
Market-making bot	Algorithmic liquidity management to keep spreads tight	$5K - $10K
Push notifications	WhatsApp / SMS / push alerts	$3K - $5K
User-generated markets	Moderated user-submitted markets	$10K - $15K
Leaderboard	Public user ranking by P&L and volume	$3K - $5K
Native mobile apps	iOS / Android via React Native or advanced PWA	$15K - $25K
Referral program	Growth incentives for invited users	$5K - $8K
Analytics dashboard	Public protocol metrics dashboard (TVL, volume, users)	$5K - $8K

---

09. Technical Stack Summary

Layer	Technology	Why it was chosen
Blockchain	Arbitrum	Low gas, EVM-compatible, native USDC, mature ecosystem
Contracts	Solidity 0.8.24 + OpenZeppelin + Foundry	Custom architecture, ERC-1155, CPMM AMM with dynamic fees
Tokens	`PronosToken` (ERC-1155)	One contract for all markets, gas efficient
AMM	`PronosAMM` (`x * y = k`)	No off-chain engine, automatic liquidity, dynamic fees
Sports oracle	Custodial multisig via Safe 2-of-3	Public results, direct control, fast resolution
Political oracle	UMA Optimistic Oracle	Industry standard, on-chain dispute process

Layer	Technology	Why it was chosen
Collateral	USDC (Circle)	Most liquid stablecoin on Arbitrum
On-ramp	MoonPay + SPEI	Apple Pay, Google Pay, cards, and MXN bank transfer
Wallet / Auth	Privy (EIP-4337 account abstraction)	Email / Google / wallet with no seed phrases
Frontend	React 18 + Vite 5	Fast iteration, SPA, mobile-first
Backend	Vercel Serverless Functions	Zero-config deployment, autoscaling, built-in cron
Database	Neon PostgreSQL	Serverless Postgres that pairs well with Vercel
Monitoring	Sentry + structured logging	Privacy-safe error tracking
Multisig	Gnosis Safe	Industry-standard treasury and admin control
Testing	Foundry (`forge test`)	43 tests, >75% coverage

---

10. Competitive Differentiators

Feature	Polymarket	Myriad Markets	Pronos
Curated LATAM markets	No	No	Yes - Liga MX, Mexico politics, culture, FIFA
Language	English	English	Native Spanish
Crypto-free onboarding	No, requires wallet and bridging	Partial	Yes - email / Google -> embedded wallet
MXN on-ramp	No	No	Yes - MoonPay + SPEI
Transparent fees	0% with hidden spread	Variable	Explicit dynamic fee (0.05% - 2.5%)
Resolution	UMA (slow)	Centralized	Fast multisig (24-48h) + UMA for politics
Target user	Global crypto traders	Global	LATAM and non-crypto users

---

11. What We Need From a Dev Shop

Executive Summary

About 75% of the product is already built. The smart contracts exist and are tested. The frontend is functional with active Polymarket mode. The backend, including indexer, APIs, and database, is operational.

What is still needed to reach mainnet:

Critical Priority (before June 14)

1. Deploy contracts to Arbitrum Sepolia, test them, then deploy to Arbitrum One
2. Connect the frontend to the internal contracts (`PronosAMM` buy / sell / redeem)
3. Wire the admin panel to contracts via Safe multisig
4. Integrate MoonPay for deposits (Apple Pay, Google Pay, cards)
5. Integrate UMA Optimistic Oracle for political markets
6. Resolve critical and high security issues (C2, C3, H1, H3, H4)
7. Run end-to-end testing of the full flow on testnet
8. Load 5 to 10 curated launch markets (World Cup, Liga MX, politics)

Medium Priority (can be post-launch)

- Gas sponsoring through Privy paymaster
- Historical price chart (OHLC)
- Unified portfolio (Polymarket + internal protocol)
- Rate limiting and input validation on APIs
- Final responsive pass
- SPEI on-ramp

Nice to Have (post-MVP)

- Categorical markets (3+ outcomes)
- Push notifications
- Leaderboard
- Market-making bot
- User-generated markets
- Native apps

Repository and Access

Resource	URL / Location
GitHub repository	`github.com/mezcalpapieth-jpg/pronos` (private)
Staging frontend	`pronos.io/mvp/`

Resource	URL / Location
Contracts	`contracts/src/`
Deploy scripts	`contracts/script/DeployProtocol.s.sol`
Serverless API	`frontend/api/`
Database	Neon PostgreSQL (connection string in Vercel env vars)
Tests	`contracts/test/` - 43 tests, run with `forge test`

Prepared by Simon Lacy - Founder, PRONOS
Last updated: April 2026
Confidential - For development evaluation only. Do not distribute.